arXiv:2310.14491v1 [cs.CL] 23 Oct 2023

Towards a Mechanistic Interpretation of Multi-Step Reasoning Capabilities of Language Models

Yifan Hou[1], **Jiaoda Li**[1], **Yu Fei**[2], **Alessandro Stolfo**[1], **Wangchunshu Zhou**[3], **Guangtao Zeng**[4], **Antoine Bosselut**[5], **Mrinmaya Sachan**[1]

[1]ETH Zürich, [2]UC Irvine, [3]AIWaves, [4]SUTD, [5]EPFL

[1]{yifan.hou, jiaoda.li, alessandro.stolfo, mrinmaya.sachan}@inf.ethz.ch, [2]yu.fei@uci.edu, [3]chunshu@aiwaves.cn, [4]guangtao_zeng@mymail.sutd.edu.sg, [5]antoine.bosselut@epfl.ch

Abstract

Recent work has shown that language models (LMs) have strong multi-step (i.e., procedural) reasoning capabilities. However, it is unclear whether LMs perform these tasks by cheating with answers memorized from pretraining corpus, or, via a multi-step reasoning mechanism. In this paper, we try to answer this question by exploring a mechanistic interpretation of LMs for multi-step reasoning tasks. Concretely, we hypothesize that the LM implicitly embeds a reasoning tree resembling the correct reasoning process within it. We test this hypothesis by introducing a new probing approach (called `MechanisticProbe`) that recovers the reasoning tree from the model's attention patterns. We use our probe to analyze two LMs: GPT-2 on a synthetic task (k-th smallest element), and LLaMA on two simple language-based reasoning tasks (ProofWriter & AI2 Reasoning Challenge). We show that `MechanisticProbe` is able to detect the information of the reasoning tree from the model's attentions for most examples, suggesting that the LM indeed is going through a process of multi-step reasoning within its architecture in many cases.[1]

1 Introduction

Large language models (LMs) have shown impressive capabilities of solving complex reasoning problems (Brown et al., 2020; Touvron et al., 2023). Yet, what is the underlying "thought process" of these models is still unclear (Figure 1). Do they *cheat with shortcuts memorized from pretraining corpus* (Carlini et al., 2022; Razeghi et al., 2022; Tang et al., 2023)? Or, do they *follow a rigorous reasoning process and solve the problem procedurally* (Wei et al., 2022; Kojima et al., 2022)? Answering this question is not only critical to our understanding of these models but is also critical for the development of next-generation faithful language-based reasoners (Creswell and Shanahan, 2022; Creswell et al., 2022; Chen et al., 2023).

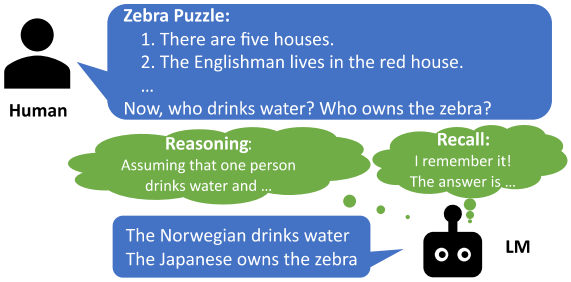

Figure 1: An example showing how LMs solve reasoning tasks. It is unclear if LMs produce the answer by reasoning or by recalling memorized information.

A recent line of work tests the behavior of LMs by designing input-output reasoning examples (Zhang et al., 2023; Dziri et al., 2023). However, it is expensive and challenging to construct such high-quality examples, making it hard to generalize these analyses to other tasks/models. Another line of work, *mechanistic interpretability* (Merullo et al., 2023; Wu et al., 2023; Nanda et al., 2023; Stolfo et al., 2023; Bayazit et al., 2023), directly analyzes the parameters of LMs, which can be easily extended to different tasks. Inspired by recent work (Abnar and Zuidema, 2020; Voita et al., 2019; Manning et al., 2020; Murty et al., 2023) that uses attention patterns for linguistic phenomena prediction, we propose an attention-based mechanistic interpretation to expose how LMs perform multi-step reasoning tasks (Dong et al., 2021).

We assume that the reasoning process for answering a multi-step reasoning question can be represented as a reasoning tree (Figure 2). Then, we investigate if the LM implicitly infers such a tree when answering the question. To achieve this, we designed a probe model, `MechanisticProbe`, that recovers the reasoning tree from the LM's attention patterns. To simplify the probing problem and gain a more fine-grained understanding, we decompose the problem of discovering reasoning trees into two

[1]Our code, as well as analysis results, are available at https://github.com/yifan-h/MechanisticProbe.

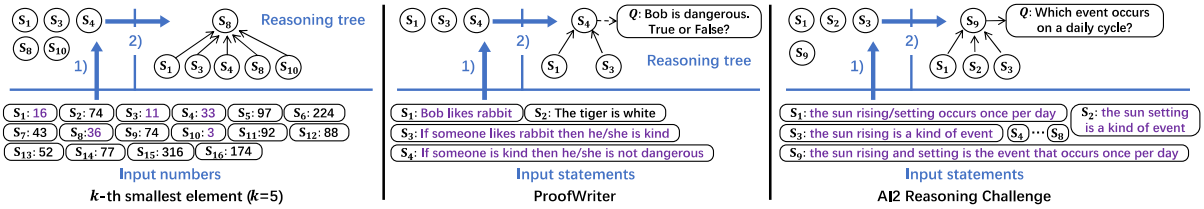

Figure 2: Illustration of our `MechanisticProbe` with one example from each of the three reasoning tasks considered in this work. We are given a number of input statements: $\mathcal{S} = \{S_1, S_2, ...,\}$ and a question: Q. `MechanisticProbe` recovers the reasoning tree describing the ground-truth reasoning process to answer the question. `MechanisticProbe` works in two stages: in the first stage, `MechanisticProbe` detects if the LM can select the set of useful statements required in reasoning, and then in the second stage, `MechanisticProbe` detects if the LM can predict the reasoning tree given the useful statements.

subproblems: 1) identifying the necessary nodes in the reasoning tree; 2) inferring the heights of identified nodes. We design our probe using two simple non-parametric classifiers for the two subproblems. Achieving high probing scores indicates that the LM captures the reasoning tree well.

We conduct experiments with GPT-2 (Radford et al., 2019) on a synthetic task (finding the k-th smallest number in a sequence of numbers) and with LLaMA (Touvron et al., 2023) on two natural language reasoning tasks: ProofWriter (Tafjord et al., 2021) and AI2 Reasoning Challenge (i.e., ARC: Clark et al., 2018). For most examples, we successfully detect reasoning trees from attentions of (finetuned) GPT-2 and (few-shot & finetuned) LLaMA using `MechanisticProbe`. We also observe that LMs find useful statements immediately at the bottom layers and then do the subsequent reasoning step by step (§4).

To validate the influence on the LM's predictions of the attention mechanisms that we identify, we conduct additional analyses. First, we prune the attention heads identified by `MechanisticProbe` and observe a significant accuracy degradation (§5). Then, we investigate the correlation between our probing scores and the LM's performance and robustness (§6). Our findings suggest that LMs exhibit better prediction accuracy and tolerance to noise on examples with higher probing scores. Such observations highlight the significance of accurately capturing the reasoning process for the efficacy and robustness of LMs.

2 Reasoning with LM

In this section, we formalize the reasoning task and introduce the three tasks used in our analysis: **k-th smallest element**, **ProofWriter**, and **ARC**.

2.1 Reasoning Formulation

In our work, the LM is asked to answer a question Q given a set of statements denoted by $\mathcal{S} = \{S_1, S_2, ...\}$. Some of these statements may not be useful for answering Q. To obtain the answer, the LM should perform reasoning using the statements in multiple steps. We assume that this process can be represented by a reasoning tree G. We provide specific examples in Figure 2 for the three reasoning tasks used in our analysis.[2] This is a very broad formulation and includes settings such as theorem proving (Loveland, 1980) where the statements could be facts or rules. In our analyses, we study the tasks described below.

2.2 Reasoning Tasks

k-th smallest element. In this task, given a list of m numbers ($m = 16$ by default) in any order, the LM is asked to predict (i.e., generate) the k-th smallest number in the list. For simplicity, we only consider numbers that can be encoded as one token with GPT-2's tokenizer. We select m numbers randomly among them to construct the input number list. The reasoning trees have a depth of 1 (Figure 2 left). The root node is the k-th smallest number and the leaf nodes are top-k numbers.

For this task, we select **GPT-2** (Radford et al., 2019) as the LM for the analysis. We randomly generate training data to finetune GPT-2 and ensure that the test accuracy on the reasoning task is larger than 90%. For each k, we finetune an independent GPT-2 model. More details about finetuning (e.g., hyperparameters) are in Appendix B.1.

ProofWriter. The ProofWriter dataset (Tafjord et al., 2021) contains theorem-proving problems.

[2]Note that we can leave out the question if Q remains the same for all examples (e.g., Figure 2 left).

In this task, given a set of statements (verbalized rules and facts) and a question, the LM is asked to determine if the question statement is true or false. Annotations of reasoning trees G are also provided in the dataset. Again, each tree has only one node at any height larger than 0. Thus, knowing the node height is sufficient to recover G. To simplify our analysis, we remove examples annotated with multiple reasoning trees. Details are in Appendix C.3. Furthermore, to avoid tree ambiguity (Appendix C.4), we only keep examples with reasoning trees of depth upto 1, which account for 70% of the data in ProofWriter.

AI2 Reasoning Challenge (ARC). The ARC dataset contains multiple-choice questions from middle-school science exams (Clark et al., 2018). However, the original dataset does not have reasoning tree annotations. Thus, we consider a subset of the dataset provided by Ribeiro et al. (2023), which annotates around 1000 examples. Considering the limited number of examples, we do not include analysis of finetuned LMs and mainly focus on the in-context learning setting. More details about the dataset are in Appendix C.3.

For both ProofWriter and ARC tasks, we select **LLaMA (7B)** (Touvron et al., 2023) as the LM for analysis. The tasks are formalized as classifications: predicting the answer token (e.g., *true* or *false* for ProofWriter).[3] We compare two settings: LLaMA with 4-shot in-context learning setting, and LLaMA finetuned with supervised signal (i.e., LLaMA_{FT}). We partially finetune LLaMA on attention parameters. Implementation details about in-context learning and finetuning on LLaMA can be found in Appendix C.1.

3 MechanisticProbe

In this section, we introduce MechanisticProbe: a probing approach that analyzes how LMs solve multi-step reasoning tasks by recovering reasoning trees from the attentions of LMs.

3.1 Problem Formulation

Our goal is to shed light on how LMs solve procedural reasoning tasks. Formally, given an LM with L layers and H attention heads, we assume that the LM can handle the multi-step reasoning task with sufficiently high accuracy.

[3]Note that most reasoning tasks with LMs can be formalized as the single-token prediction format (e.g., multi-choice question answering).

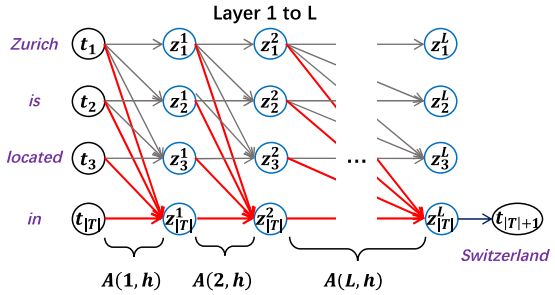

Figure 3: An illustration of the attention mechanism in a LM. The example sentence here is: *Zurich is located in Switzerland*. Token $t_{|T|}$ is the last token used for the prediction of the next token $t_{|T|+1}$. The arrows here show the attentions, where the red arrows denote the attentions to the last token.

Let us consider the simplest of the three tasks (k-th smallest element). Each statement S_i here is encoded as one token t_i. The input text (containing all statements in $\mathcal{S}$) to the LM comprises a set of tokens as $T = (t_1, t_2, ...)$. We denote the hidden representations of the token t_i at layer l by $\boldsymbol{z}_i^l$. We further denote the attention of head h between $\boldsymbol{z}_i^{l+1}$ and $\boldsymbol{z}_j^l$ as $\boldsymbol{A}(l,h)[i,j]$. As shown in Figure 3, the attention matrix at layer l of head h is denoted as $\boldsymbol{A}(l,h)$, the lower triangular matrix.[4] The overall attention matrix then can be denoted by $\boldsymbol{A} = \{\boldsymbol{A}(l,h) | 1 \leq l \leq L; 1 \leq h \leq H\}$.

Our probing task is to detect G from $\boldsymbol{A}$, i.e. modeling $P(G|\boldsymbol{A})$. However, note that the size of $\boldsymbol{A}$ is very large – $L \times H \times |T|^2$, and it could contain many redundant features. For example, if the number of tokens in the input is 100, the attention matrix $\boldsymbol{A}$ for LLaMA contains millions of attention weights. It is impossible to probe the information directly from $\boldsymbol{A}$. In addition, the tree prediction task is difficult (Hou and Sachan, 2021) and we want our probe to be simple (Belinkov, 2022) as we want it to provide reliable interpretation about the LM rather than learn the task itself. Therefore, we introduce two ways to simplify attentions and the probing task design.

3.2 Simplification of $\boldsymbol{A}$

In general, we propose two ways to simplify $\boldsymbol{A}$. Considering that LLaMA is a large LM, we propose extra two ways to further reduce the number of considered attention weights in $\boldsymbol{A}$ for it.

Focusing on the last token. For causal LMs, the representation of the last token in the last layer $\boldsymbol{z}_{|T|}^L$

[4]Note that in this work we only consider causal LMs.